ASSIGNMENT-13.1

HT NO:2303A51804

Batch No:28

Course: AI Assisted Coding

Task Description 1 : Refactoring – Removing Code Duplication

• Task: Use AI to refactor a given Python script that contains

multiple repeated code blocks.

• Instructions:

o Prompt AI to identify duplicate logic and replace it

with functions or classes.

o Ensure the refactored code maintains the same output.

o Add docstrings to all functions.

• Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Prompt:

Refactor the following Python code to remove duplicate logic. Identify repeated code blocks and replace them with a reusable function. Ensure the output remains the same and add appropriate docstrings to the function.

Code:

```python
#Refactor the following Python code to remove duplicate logic. Identify repeated code blocks and replac
#Code:
#print("Area of Rectangle:", 5 * 10)
#print("Perimeter of Rectangle:", 2 * (5 + 10))
#print("Area of Rectangle:", 7 * 12)
#print("Perimeter of Rectangle:", 2 * (7 + 12))
#print("Area of Rectangle:", 10 * 15)
#print("Perimeter of Rectangle:", 2 * (10 + 15))
def calculate_rectangle_properties(length, width):
    """
    Calculate and print the area and perimeter of a rectangle.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    None
    """
    area = length * width
    perimeter = 2 * (length + width)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")
# Calculate properties for different rectangles
calculate_rectangle_properties(5, 10)
calculate_rectangle_properties(7, 12)
calculate_rectangle_properties(10, 15)
```

Output:

```
AI_Assisant_Coding/program13.py"
AI_Assisant_Coding/program13.py"
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
(.venv) PS E:\Learning Courses\AI_Assisant_Coding>
```

Justification:

1. The original code had repeated calculations for rectangle area and perimeter.

2. Duplicate code makes programs harder to maintain.

3. A reusable function was created to handle the calculations.

4. This reduces repetition and improves code organization.

5. Any future changes can be done in one place.

6. Docstrings were added to explain the function clearly.

Task Description 2 :(Refactoring – Optimizing Loops andConditionals)

• Task: Use AI to analyze a Python script with nested loops

and complex conditionals.

• Instructions:

o Ask AI to suggest algorithmic improvements (e.g.,

replace nested loops with set lookups or

comprehensions).

o Implement changes while keeping logic intact.

o Compare execution time before and after refactoring.

• Sample Legacy Code:

```
# Legacy inefficient code
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
found = False
for n in names:
if s == n:
found = True
if found:
```

print(f"{s} is in the list")

else:

print(f"{s} is not in the list")

Prompt:

Analyze the following Python code that uses nested loops to search for names in a list. Suggest an optimized approach using efficient data structures such as sets or list comprehensions. Refactor the code while keeping the same functionality and compare execution time before and after optimization.

Python Code:

```
#Analyze the following Python code that uses nested loops to search for names in a list. Suggest an optimized approac

#Code:
#names = ["Alice", "Bob", "Charlie", "David"]
#search_names = ["Charlie", "Eve", "Bob"]

#for s in search_names:
    #found = False
    #for n in names:
       # if s == n:
        #    found = True

    #if found:
        #print(f"{s} is in the list")
    #else:
       # print(f"{s} is not in the list")
# Optimized approach using sets
import time
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
# Convert names list to a set for O(1) average time complexity lookups
names_set = set(names)
start_time = time.time()
```

```python
# Optimized approach using sets
import time
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
# Convert names list to a set for O(1) average time complexity lookups
names_set = set(names)
start_time = time.time()
for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
end_time = time.time()
print(f"Execution time after optimization: {end_time - start_time:.6f} seconds")
# Refactored code using list comprehensions
start_time = time.time()
results = [f"{s} is in the list" if s in names_set else f"{s} is not in the list" for s in search_names]
for result in results:
    print(result)
end_time = time.time()
print(f"Execution time after refactoring: {end_time - start_time:.6f} seconds")
```

Output:

```
(.venv) PS E:\Learning Courses\AI_Assisant_Coding> & "e:\Learning
AI_Assisant_Coding/program13.py"
Charlie is in the list
Eve is not in the list
Bob is in the list
Execution time after optimization: 0.001085 seconds
Charlie is in the list
Eve is not in the list
Bob is in the list
Execution time after refactoring: 0.000845 seconds
(.venv) PS E:\Learning Courses\AI_Assisant_Coding>
```

Justification:

1. The legacy code used nested loops to search for names.

2. Nested loops increase time complexity and slow down execution.

3. A set was used for faster membership checking.

4. Set lookup works in constant time O(1).

5. This significantly improves performance for large datasets.

6 Execution time comparison shows the optimization effect.

Task Description 3: Refactoring – Extracting Reusable Functions

• Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

• Instructions:

o Identify repeated or related logic and extract it into reusable functions.

o Ensure the refactored code is modular, easy to read, and documented with docstrings.

• Sample Legacy Code:

```
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Prompt:

Refactor the following Python code by extracting the repeated tax and total price calculation into a reusable function called calculate_total(price). Ensure the code is modular, readable, and includes proper docstrings.

Python Code:

```
#Refactor the following Python code by extracting the repeated tax and total price calculation into a reusable functi

#Code:
#price = 250
#tax = price * 0.18
#total = price + tax
#print("Total Price:", total)
#price = 500
#tax = price * 0.18
#total = price + tax
#print("Total Price:", total)
def calculate_total(price):
    """
    Calculate the total price including tax.

    Args:
        price (float): The original price of the item.

    Returns:
        float: The total price after adding tax.
    """
    tax = price * 0.18
    total = price + tax
    return total
# Example usage
price1 = 250
total1 = calculate_total(price1)
print("Total Price for price1:", total1)
price2 = 500
```

```
    """
    tax = price * 0.18
    total = price + tax
    return total
# Example usage
price1 = 250
total1 = calculate_total(price1)
print("Total Price for price1:", total1)
price2 = 500
total2 = calculate_total(price2)
print("Total Price for price2:", total2)
```

Output:

```
(.venv) PS E:\Learning Courses\AI_Assisant_Coding> & "e:\Learn
AI_Assisant_Coding/program13.py"
Total Price for price1: 295.0
Total Price for price2: 590.0
Total Price for price2: 590.0
(.venv) PS E:\Learning Courses\AI_Assisant_Coding>
```

Justification:

1. The tax calculation logic was repeated in the program.
2. Repeated logic was moved into a function called calculate_total().
3. Functions improve modularity and code reuse.
4. Future changes (like tax rate updates) can be made easily.
5. The code becomes more structured and readable.
6. A docstring was added to describe the function.

Task Description 4: Refactoring – Replacing Hardcoded Values with Constants

• Task: Use AI to identify and replace all hardcoded “magic

numbers” in the code with named constants.

• Instructions:

o Create constants at the top of the file.

o Replace all hardcoded occurrences in calculations

with these constants.

o Ensure the code remains functional and is easier to

maintain.

• Sample Legacy Code:

Legacy script with hardcoded values

print("Area of Circle:", 3.14159 * (7 ** 2))

print("Circumference of Circle:", 2 * 3.14159 * 7)

Prompt:

Refactor the following Python code by replacing hardcoded values (magic numbers) with named constants. Define the constants at the top of the file and update the calculations accordingly.

Python Code

```
#Refactor the following Python code by replacing hardcoded values (magic numbers) with named con

#Code:
#print("Area of Circle:", 3.14159 * (7 ** 2))
#print("Circumference of Circle:", 2 * 3.14159 * 7)
# Refactored code with named constants
# Define constants
PI = 3.14159
RADIUS = 7
# Calculate area and circumference using constants
area_of_circle = PI * (RADIUS ** 2)
circumference_of_circle = 2 * PI * RADIUS
# Print results
print("Area of Circle:", area_of_circle)
print("Circumference of Circle:", circumference_of_circle)
```

Output:

```
(.venv) PS E:\Learning Courses\AI_Assisant_Coding> & "e:\Learn
AI_Assisant_Coding/program13.py"
Area of Circle: 153.93791
Circumference of Circle: 43.98226
(.venv) PS E:\Learning Courses\AI_Assisant_Coding>
```

Justification:

1. The original code used hardcoded numbers like π and radius.

2. Hardcoded values reduce flexibility and readability.

3. Named constants such as PI and RADIUS were created.

4. Constants make the code easier to understand.

5. Changes can be made easily by modifying the constant value.

6. This improves maintainability and prevents errors.

Task Description 5: Refactoring – Improving Variable Naming and Readability

• Task: Use AI to improve readability by renaming unclear

variables and adding inline comments.

• Instructions:

o Replace vague names with meaningful ones.

o Add comments where logic is not obvious.

o Keep functionality exactly the same.

• Sample Legacy Code:

```
# Legacy script with poor variable names
a = 10
b = 20
c = a * b / 2
print(c)
```

Prompt:

Refactor to optimal and Improve the readability of the following Python code by renaming unclear variables with meaningful names and adding comments where necessary. Do not change the functionality of the program.

Python Code:

```python
#refactor to optimal and Improve the readability of the following Python code by renaming

#Code:
#a = 10
#b = 20
#c = a * b / 2
#print(c)
# Define the base and height of the triangle
base_length = 10
height_length = 20
# Calculate the area of the triangle using the formula: (base * height) / 2
triangle_area = (base_length * height_length) / 2
# Print the calculated area of the triangle
print(triangle_area)
```

Output:

```
(.venv) PS E:\Learning Courses\AI_Assisant_Coding> & e:\Learning Co
AI_Assisant_Coding/program13.py"
100.0
(.venv) PS E:\Learning Courses\AI_Assisant_Coding>
```

Justification:

1. The original variables a, b, and c were unclear.
2. Descriptive names like base, height, and area_of_triangle were used.
3. Meaningful names help developers understand the code quickly.
4. Comments were added to explain the calculation.
5. This improves code readability and maintainability.
6. The functionality of the program remains unchanged.

Task Description 6: Refactoring – Removing Redundant Conditional Logic

• Task: Use AI to refactor a Python script that contains repeated if–else

logic for grading students.

• Instructions:

o Ask AI to identify redundant conditional checks.

o Replace them with a reusable function.

o Ensure grading logic remains unchanged.

• Code:

```
marks = 85

if marks >= 90:

print("Grade A")

elif marks >= 75:

print("Grade B")

else:

print("Grade C")

marks = 72

if marks >= 90:

print("Grade A")

elif marks >= 75:

print("Grade B")

else:

print("Grade C")
```

Prompt:

Refactor the following Python code to remove redundant grading logic. Create a reusable function called calculate_grade(marks) that determines the grade based on marks. Ensure the grading logic remains unchanged and add a docstring to the function.

Python Code:

```python
#Refactor the following Python code to remove redundant grading logic. Create a reusable function ca
#Code:
#marks = 85
#if marks >= 90:
   # print("Grade A")
#elif marks >= 75:
   # print("Grade B")
#else:
    #print("Grade C")
#marks = 72
#if marks >= 90:
   # print("Grade A")
#elif marks >= 75:
   # print("Grade B")
#else:
   # print("Grade C")
def calculate_grade(marks):
    """
    Determines the grade based on the given marks.

    Parameters:
    marks (int): The marks obtained by the student.

    Returns:
    str: The grade corresponding to the marks.
    """
    if marks >= 90:
        return "Grade A"
    elif marks >= 75:
```

```
    marks (int): The marks obtained by the student.

    Returns:
    str: The grade corresponding to the marks.
    """
    if marks >= 90:
        return "Grade A"
    elif marks >= 75:
        return "Grade B"
    else:
        return "Grade C"
# Example usage
marks1 = 85
marks2 = 72
print(calculate_grade(marks1))  # Output: Grade B
print(calculate_grade(marks2))  # Output: Grade C
```

Output:

```
AI_Assisant_Coding/program13.py"
Grade B
Grade C
(.venv) PS E:\Learning Courses\AI_Assisant_Coding>
```

Justification:

1. The grading logic was repeated multiple times.
2. Repeated if–elif–else conditions increase code duplication.
3. A reusable function calculate_grade() was created.
4. The function centralizes the grading logic.
5. Future grading rule changes become easier to update.
6. A docstring improves documentation of the function.

Task Description 7: Refactoring – Converting Procedural Code to Functions

• Task: Use AI to refactor procedural input–processing logic into functions.

• Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function.

o Improve code readability without changing behavior.

• Sample Legacy Code:

```
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
```

Prompt:

Refactor the following procedural Python code into modular functions. Separate the program into input, processing, and output functions such as get_input(), calculate_square(), and display_result(). Ensure the functionality remains the same.

Python Code

```python
#Refactor the following procedural Python code into modular functions. Separate the program into input,
#Code:
#num = int(input("Enter number: "))
#square = num * num
#print("Square:", square)
def get_input():
    num = int(input("Enter number: "))
    return num
#def calculate_square(num):
def calculate_square(num):
    square = num * num
    return square
def display_result(square):
    print("Square:", square)
def main():
    num = get_input()
    square = calculate_square(num)
    display_result(square)
if __name__ == "__main__":
    main()
```

Output:

```
AI_Assisant_Coding/program13.py"
Enter number: 5
Square: 25
(.venv) PS E:\Learning Courses\AI_Assisant_Coding>
```

Justification:

1. The original program handled input, processing, and output together.
2. This makes the code less organized.
3. The code was divided into separate functions.
4. Functions improve modularity and readability.
5. Each function now has a clear responsibility.
6. This structure makes debugging and testing easier.

Task Description 8 :Refactoring – Optimizing List Processing

• Task: Use AI to refactor inefficient list processing logic.

• Instructions:

o Ask AI to replace manual loops with list comprehensions or built-in

functions.

o Ensure output remains identical.

• Sample Legacy Code:

nums = [1, 2, 3, 4, 5]

squares = []

for n in nums:

squares.append(n * n)

print(squares)

Prompt:

Refactor the following Python code to improve efficiency and readability. Replace the manual loop used for list processing with a Python list comprehension while keeping the output the same.

Python Code

```
#Refactor the following Python code to improve efficiency and readability. Replace the manual
#Code:
#nums = [1, 2, 3, 4, 5]
#squares = []
#for n in nums:
    #squares.append(n * n)
#print(squares)
nums = [1, 2, 3, 4, 5]
squares = [n * n for n in nums]
print(squares)
```

Output:

```
(.venv) PS E:\Learning Courses\AI_Assisant_Coding> & "e:\Learning Courses\AI_A
AI_Assisant_Coding/program13.py"
[1, 4, 9, 16, 25]
(.venv) PS E:\Learning Courses\AI_Assisant_Coding>
```

Justification:

1. The legacy code used a manual loop to build a list of squares.

2. Manual loops increase the number of lines of code.

3. List comprehension was used for a cleaner solution.

4. It makes the code shorter and easier to read.

5. List comprehensions are often faster in Python.

6. The output remains exactly the same.